EDUCONECTA

Guia de Arquitetura e Handoff Técnico

Documento de passagem de testemunho: o essencial para quem pegar no projeto e continuar o trabalho sem redescobrir as decisões já tomadas. Cobre a arquitetura, as duas fases já implementadas (gestão de faltas/justificações e notificações via RabbitMQ) e como pôr tudo a correr.

Estado atual: 6 microsserviços + RabbitMQ a correr em Docker Compose, todos ligados ao MongoDB Atlas, com 143 testes automatizados (pytest) a passar. Fluxo completo login → criar dados → persistir na Atlas → publicar/consumir eventos RabbitMQ testado manualmente ponta a ponta.

EduConecta é uma aplicação de gestão escolar — comunicação entre a escola, encarregados de educação e profissionais (docentes, educadoras, auxiliares). É construída como microsserviços independentes em Python/Flask (o **auth** é assíncrono, em Quart), organizados em MVC, todos ligados ao mesmo cluster MongoDB Atlas na cloud, com RabbitMQ para comunicação assíncrona entre serviços.

1. Arquitetura geral

O cliente fala diretamente com cada serviço (não há gateway/API única). Dentro da rede Docker, os serviços de negócio consultam o **auth** para validar a sessão de quem fez o pedido; o **comunicados**/**assiduidade**/**diario** consultam também o **escolar** para confirmar relações (ex: este encarregado é mesmo encarregado deste aluno? esta turma é do tipo creche?). Desde a Fase 2, três serviços publicam eventos no **RabbitMQ** e o novo serviço **notificacoes** consome-os em segundo plano.

Serviço	Porta	Trata de
auth (assíncrono)	5002	Contas, login, JWT, perfis, criação de contas pela administração, envio de credenciais por email.
escolar	5001	Alunos, turmas (incl. tipo creche/básico), encarregados de educação, profissionais e associações. Também expõe o endpoint interno de resolução de destinatários para o notificacoes.
diario	5003	Registos diários de acompanhamento de cada aluno — só disponível para turmas do tipo **creche**.
comunicados	5004	Comunicados (criar/editar/eliminar/listar) com fotografias, e chat com fotografias e vídeos.
assiduidade	5005	Registo de presenças/faltas por turma, e o fluxo completo de justificação de faltas (texto e/ou anexo).
notificacoes (Fase 2)	5006	Consome eventos do RabbitMQ e grava notificações in-app por utilizador (GET /me/notificacoes).
rabbitmq (Fase 2)	5672 / 15672	Broker de mensagens (AMQP) + painel de gestão web em :15672.

Linhas sólidas seriam os pedidos do cliente a cada serviço; **linhas finas** seriam a comunicação interna entre serviços (verificação de sessão/permissões, resolução de relações) e a ligação de cada serviço à Atlas. Os três serviços publicadores (escolar, comunicados, assiduidade) enviam eventos para o exchange do RabbitMQ; o notificacoes consome-os de uma fila própria.

2. Autenticação e permissões

Login é feito só no **auth** (`POST /login`), que devolve um JWT válido por 15 minutos. Esse token vai no cabeçalho `Authorization: Bearer <token>` em todos os pedidos seguintes, a qualquer um dos serviços (exceto o endpoint interno do notificacoes/escolar, ver secção 6).

Não há registo público — só uma conta **admin** pode criar outras contas (`POST /admin/utilizadores`, ou automaticamente ao criar um encarregado/profissional em **escolar**). As contas novas recebem uma password aleatória por email e são obrigadas a trocá-la no primeiro acesso (`deve_alterar_password`).

Perfis existentes

- **admin** — gestão completa; cria/edita/desativa/elimina contas, turmas, alunos.
- **encarregado** — consulta dados dos seus educandos, submete justificações de faltas, usa o chat.

- **docente** ("professor") — no máximo 1 por turma (é sempre, de facto, quem a escola trataria como "diretor de turma", mas **não existe esse campo/conceito no modelo** — ver secção 4).
- **educadora** — comunicados, chat, registo diário, regista falta, aprova/rejeita justificações.
- **auxiliar** — apenas registo diário.

Cada rota decide quais perfis aceita (normalmente: admin sempre pode; outros perfis só nas rotas pessoais, ex: `/me/alunos` para encarregados). O soft delete (`estado: "eliminado"`) é usado para utilizadores e alunos — o documento fica na base de dados mas bloqueado (login, listagens, tudo falha), para não deixar outros serviços a apontar para um id que desapareceu.

3. Estrutura de código (MVC)

A mesma estrutura repete-se em todos os serviços síncronos — só muda o conteúdo de `models/` e `controllers/`.

```
<servico>/
app.py cria a app, regista blueprints, autenticação global
db.py liga ao MongoDB Atlas
openapi.yaml especificação OpenAPI (Swagger em /docs)
models/ acesso a dados (queries MongoDB), sem HTTP
controllers/ rotas + validação + permissões
utils/ tokens, chamadas a outros serviços, RabbitMQ
tests/ pytest (BD e serviços externos mockados)
```

Regra de ouro ao mexer no código

- Lógica de acesso a dados (queries MongoDB) vive em `models/`, nunca direto no controller.
- Validação de input, permissões e formato da resposta vivem em `controllers/`.
- Chamadas a outros serviços (HTTP interno, RabbitMQ) vivem em `utils/`.
- `app.py` só cria a app e regista rotas/hooks globais — nunca cresce com lógica de negócio.

Nota: o **auth** foi o piloto de uma migração para assíncrono (Quart + PyMongo AsyncMongoClient + aiosmtplib). Os outros 5 serviços (incluindo o novo notificacoes) ficaram em Flask síncrono por decisão de âmbito — dá para converter depois, seguindo o mesmo padrão do auth como referência.

4. Fase 1 — gestão de alunos, turmas e faltas

A Fase 1 fechou vários requisitos funcionais em falta. O plano completo, com as decisões e o porquê de cada uma, está em `docs/fase-1-plano.md`. Pontos a reter:

Turmas: tipo e sem "diretor de turma"

- Toda a turma tem um campo `tipo`: **creche** (aceita diário e faltas) ou **basico** (só aceita faltas — o serviço diario recusa com 409 `class_type_not_supported` para turmas básico).
- Uma turma só pode ter **1 docente** atribuído de cada vez (409 `class_already_has_teacher` ao tentar associar um segundo); educadora/auxiliar não têm esse limite.
- **Não existe** o conceito de "diretor de turma" como campo/relação — qualquer docente ou educadora atribuído a uma turma pode aprovar/rejeitar justificações de falta dessa turma. Foi uma simplificação deliberada pedida explicitamente.

Justificação de faltas (RF18, RF20, RF21, RF22)

- O encarregado submete a justificação de uma falta (`POST .../faltas/<dia>/justificacao`), com texto e/ou um anexo (até 5MB, guardado em GridFS).
- Prazo de 3 dias úteis a contar da falta — validado no momento da submissão.
- Um docente/educadora atribuído à turma aprova ou rejeita (`.../aprovar` ou `.../rejeitar`, este último exige motivo_rejeicao).

Outras alterações

- Eliminar aluno (`DELETE /alunos/<id>`) é soft delete, admin apenas.
- Um Profissional também pode editar um aluno, mas só se estiver atribuído à turma do aluno.
- Criar encarregado/profissional (em **escolar**) passou a exigir nome, email, CC (+ telemóvel opcional) e cria a conta de login automaticamente no auth — com password aleatória enviada por email, igual ao fluxo já existente para admin.
- Comunicados passaram a poder ser editados/eliminados pelo autor; o anexo de vídeo saiu dos comunicados e passou a ser suportado nas mensagens de chat (que também aceitam fotografia).

5. Fase 2 — RabbitMQ e notificações

A Fase 2 acrescentou comunicação assíncrona entre serviços. O plano completo está em `docs/fase-2-plano.md`.

Desenho de mensagens

Um único exchange **topic**: `educonecta.eventos`. Cada serviço publica com uma routing key própria; o **notificacoes** liga uma fila durável (`notificacoes`) a todas elas.

Evento (routing key)	Publicado por	Consumido para
aluno.eliminado	escolar	Só auditoria (coleção `auditoria` do notificacoes) — não há destinatário específico.

Evento (routing key)	Publicado por	Consumido para
comunicado.criado	comunicados	Notifica os encarregados dos alunos da turma do comunicado.
falta.justificacao.submetida	assiduidade	Notifica os profissionais (docente/educadora) atribuídos à turma.
falta.justificacao.decidida	assiduidade	Notifica o encarregado que submeteu a justificação.

Publicação best-effort

Cada serviço publicador tem um `utils/eventos.py` com uma função `publicar(routing_key, payload)` usando **pika**. Se o RabbitMQ estiver em baixo, o evento é só perdido (fica um aviso no log) — **nunca** faz falhar o pedido HTTP principal. Publicar um comunicado, por exemplo, nunca pode falhar só porque o RabbitMQ está indisponível.

O consumidor e o endpoint interno

O **notificacoes** arranca um consumidor numa thread de fundo (uma por worker do gunicorn — padrão "competing consumers", o RabbitMQ reparte as mensagens, nenhuma é processada duas vezes). Religa automaticamente se a ligação cair. Não há fila de mensagens mortas nem re-tentativas: se o processamento de uma mensagem falhar, fica só um aviso no log e a mensagem é descartada — decisão de âmbito aceite para um projeto académico.

Para saber *quem* notificar numa turma, o consumidor chama um endpoint **interno** novo em escolar: `GET /internal/turmas/<id>/destinatarios`. Como o consumidor corre em segundo plano, sem nenhum pedido HTTP associado, não há token de utilizador para reencaminhar — por isso este endpoint usa uma **chave partilhada** (cabeçalho `X-Internal-Key`, variável `INTERNAL_SERVICE_KEY`) em vez da autenticação normal por sessão. É a **única** exceção a "toda a comunicação interna reenvia o token do utilizador" neste projeto — vale a pena lembrar se for feita uma auditoria de segurança mais tarde.

Rotas do notificacoes

Rota	Perfil	O que faz
GET /me/notificacoes	qualquer autenticado	Lista as notificações do próprio utilizador, paginado (20/página).
PATCH /me/notificacoes/<id>/lida	dono da notificação	Marca como lida.

Âmbito deliberadamente fora da Fase 2: notificações só in-app (sem email — o envio de credenciais por email no auth mantém-se, isso é outra coisa); sem dead-letter queue/retries; sem deploy cloud do RabbitMQ (RNF09/10, continua adiado para o fim do projeto, só se sobrar tempo).

6. Base de dados (MongoDB Atlas)

Todos os serviços ligam ao **mesmo cluster Atlas**, cada um com a sua própria base de dados lá dentro — não há isolamento de utilizador por serviço (um único utilizador admin do Atlas, decisão deliberada para simplificar um projeto académico). As bases de dados e coleções são criadas automaticamente na primeira escrita.

Base de dados	Serviço	Principais coleções
auth_db	auth	utilizadores
escolar_dc	escolar	alunos, turmas, encarregados_educacao, profissionais
diario_db	diario	registos
comunicados_db	comunicados	comunicados, conversas, mensagens (+ GridFS fotos/vídeos)
educonecta_assiduidade	assiduidade	assiduidade, justificacoes (+ GridFS anexos)
educonecta_notificacoes	notificacoes	notificacoes, auditoria

Onde estão as credenciais: não estão neste documento, de propósito. Estão em dois ficheiros na raiz do projeto (`EduConecta_projeto/`), ambos fora do git (`.gitignore`): `.env` (variáveis usadas pelo Docker Compose — Mongo, JWT, RabbitMQ, chave interna) e `.env.smtp` (SMTP_USERNAME/SMTP_PASSWORD, envio de email). Pede-os diretamente a quem já tem o projeto a correr — nunca os reenvies por email/chat sem necessidade.

7. Como pôr o projeto a correr

1. Variáveis de ambiente

Criar `.env` na raiz com as variáveis abaixo, e `.env.smtp` com SMTP_USERNAME/SMTP_PASSWORD.

Variável	Para quê
MONGO_URI	Connection string da Atlas (partilhada por todos os serviços)
MONGO_AUTH_DATABASE / MONGO_ESCOLAR_DATABASE / ...	Nome da BD de cada serviço (uma variável por serviço)
JWT_SECRET_KEY	Chave de assinatura dos tokens (auth), mínimo 32 caracteres
RABBITMQ_USER / RABBITMQ_PASS	Credenciais do broker RabbitMQ
INTERNAL_SERVICE_KEY	Chave partilhada do endpoint interno escolar ↔ notificacoes
SMTP_USERNAME / SMTP_PASSWORD	Em .env.smtp — envio de credenciais por email (auth)

2. Arrancar tudo

A partir de `EduConecta_projeto/`:

```
docker compose up --build -d
docker compose ps
```

```
curl http://127.0.0.1:5002/health
```

3. Portas

escolar :5001 · auth :5002 · diario :5003 · comunicados :5004 · assiduidade :5005 · notificacoes :5006 · rabbitmq :5672 (AMQP) e :15672 (painel web). Todos os serviços HTTP têm `/health` e `/ready` (este último confirma a ligação à Atlas).

4. Explorar a API

Cada serviço tem a sua própria documentação Swagger em `/docs` (ex: `http://localhost:5001/docs`) e a especificação crua em `/openapi.yaml` — é a fonte mais fiável e atualizada de cada endpoint, parâmetros e códigos de erro, mais detalhada do que este documento.

Fluxo típico: `POST /login` no auth (:5002) com email + password → guarda o `access_token` → usa esse token como Bearer Token nos pedidos aos outros serviços.

5. Correr os testes automáticos

Por serviço, a partir de `backend/services/<servico>/`:

```
pip install -r requirements-dev.txt
pytest -q
```

Estado atual: 143 testes a passar no total (auth 23, escolar 41, diario 10, comunicados 27, assiduidade 24, notificacoes 18).

6. Scripts de teste manual

A pasta `testes_manuais/` na raiz tem scripts Python avulsos (`teste_*.py`) usados para smoke-testing manual contra o Docker/Atlas reais durante o desenvolvimento — não fazem parte da suite pytest, são só utilitários de diagnóstico.

8. O que falta / pontos em aberto

Lista honesta do que não está feito, para quem continuar não perder tempo a descobrir:

- **Async nos outros 5 serviços** — só o auth foi convertido para Quart/async; escolar, diario, comunicados, assiduidade e notificacoes continuam em Flask síncrono.
- **Sem frontend** — decisão explícita do âmbito do projeto, não vai existir.
- **RNF09/RNF10 (deploy cloud, escalabilidade/Kubernetes)** — adiado para o fim, só se sobrar tempo. O diagrama-alvo do enunciado (API Gateway incluído) ainda não foi implementado; hoje o cliente fala diretamente com cada serviço.
- **Notificações só in-app** — sem email para os eventos da Fase 2 (o email de credenciais no auth é uma coisa à parte e mantém-se).
- **Sem dead-letter queue nem re-tentativas no RabbitMQ** — uma mensagem cujo processamento falhe é descartada, não fica presa nem é reprocessada.
- **Um único utilizador Atlas para tudo** — simplificação deliberada; separar por serviço é possível mais tarde se for preciso reforçar isolamento.
- **Sem CI** — os testes correm manualmente; não há pipeline a correr pytest automaticamente a cada alteração.

9. Onde procurar mais informação

- `docs/fase-1-plano.md` — decisões e âmbito completo da Fase 1 (faltas, justificações, tipo de turma, soft delete).
- `docs/fase-2-plano.md` — decisões e âmbito completo da Fase 2 (RabbitMQ).
- `/docs` (Swagger UI) em cada serviço — referência completa e sempre atualizada de cada endpoint.
- `backend/services/<servico>/tests/` — os testes automatizados são também a melhor documentação viva do comportamento esperado de cada regra de negócio.

Documento gerado a partir do estado do repositório em 2026-10-29, após a conclusão da Fase 2.